

A Modular Architecture for Decentralized Peer-to- Peer Overlay Networks

Supporting secure and privacy-oriented
communication,
data persistence, and decentralized applications

Anderson Simioni Assunção
Orientador: Carlos Becker Westphall



O problema não é apenas descentralizar

Uma rede descentralizada real precisa resolver conectividade, descoberta, segurança, privacidade e persistência.

Centralização

Serviços concentram identidade, dados, mediação de comunicação e pontos de controle.

P2P real

A rede precisa lidar com descoberta de pares, NAT traversal, rotas, churn e heterogeneidade.

Segurança futura

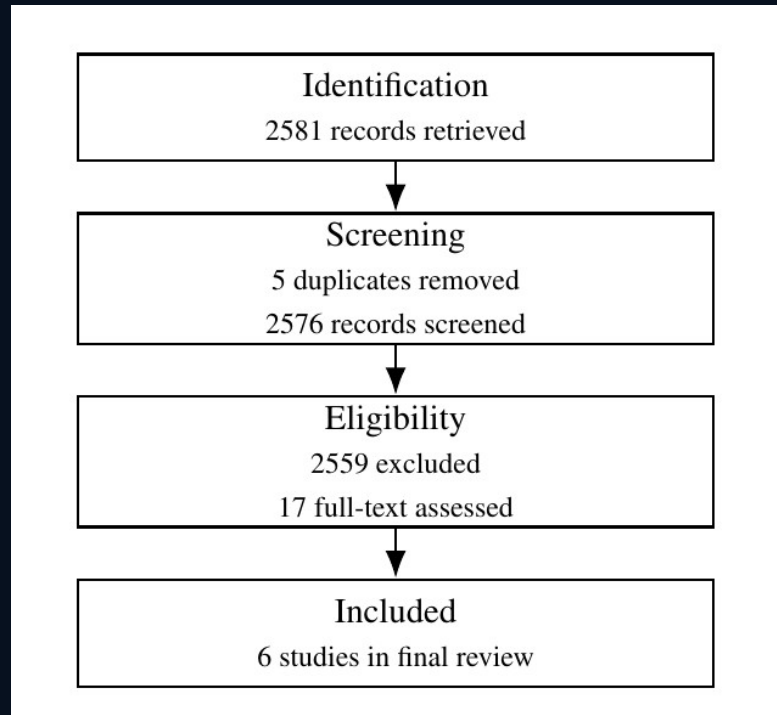
Sistemas persistentes exigem criptografia flexível e preparação para ameaças pós-quânticas.

Descentralização sem arquitetura vira apenas distribuição de complexidade.

Problema: descentralizar também cria complexidade

Revisão sistemática e lacuna encontrada

O conjunto final foi pequeno porque exigia uma interseção técnica muito específica.



PRISMA: 2581 registros -> 6 estudos

2581
registros
recuperados

17
full-text
avaliados

6
estudos
finais

Lacuna

Tor/I2P, IPFS, DHTs e PQC resolvem partes diferentes do problema. A lacuna está na integração arquitetural desses eixos em uma base modular comum.

- P2P/decentralização
- Privacidade/anonymity-oriented routing
- Segurança e criptografia
- Roteamento, descoberta, NAT e persistência

Lacuna: integração, não falta de peças

Pergunta de pesquisa e objetivo

A apresentação gira em torno de uma pergunta arquitetural.

04 · ATO 1 · OBJETIVO

Como estruturar uma camada overlay P2P capaz de combinar comunicação segura, orientação à privacidade, descoberta, roteamento, persistência e suporte a aplicações?

Projetar
arquitetura modular

Implementar
MVP experimental

Validar
fluxos principais

Demonstrar
PoC social

Escopo de validação

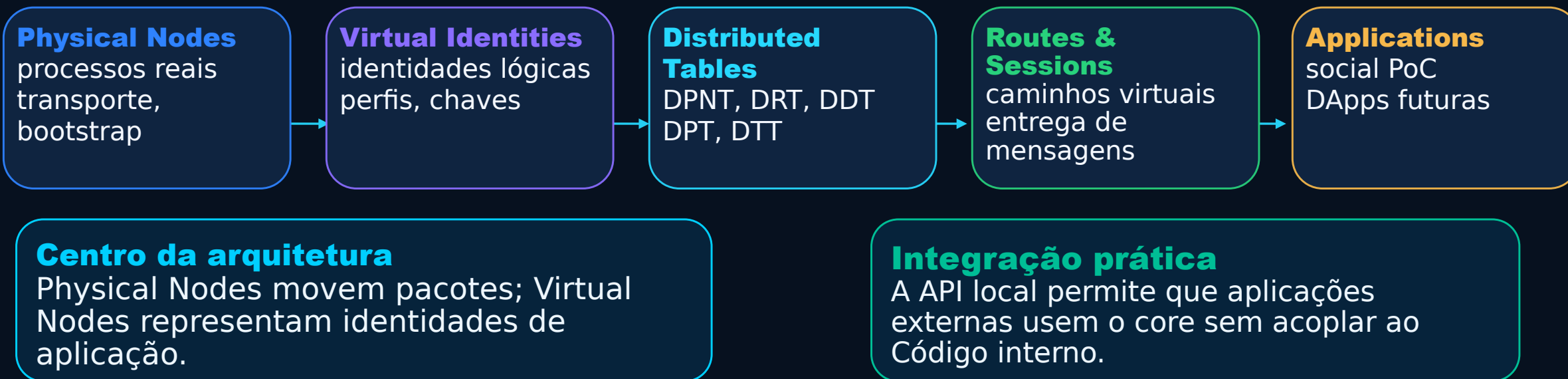
Arquitetura + implementação experimental + fluxos principais. A defesa não vende benchmark Internet-scale.

Pergunta: como organizar a camada?

Contribuição central

05 · ATO 2 · CONTRIBUIÇÃO

Separar explicitamente conexão física, identidade lógica, descoberta distribuída, rotas, sessões e aplicação.



A contribuição não é reproduzir Tor, IPFS ou Kademlia; é integrar ideias desses domínios em uma arquitetura modular e validável.

Contribuição: separar responsabilidades

Arquitetura em camadas

A modularidade permite trocar estratégias de conexão, armazenamento e criptografia sem redesenhar toda a rede.

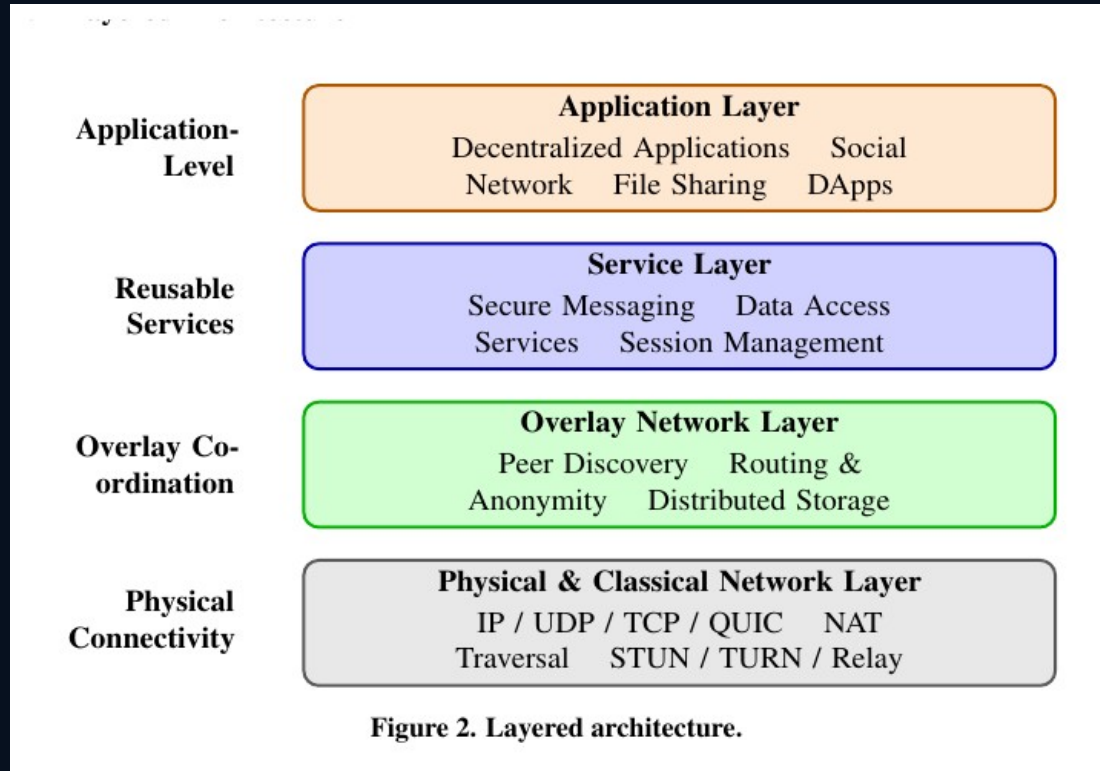


Figura 2 do TCC: arquitetura em camadas

Mensagem principal

A arquitetura é modular, mas é apresentada em camadas para separar responsabilidades: conectividade física, coordenação overlay, serviços reutilizáveis e aplicações.

- Physical layer: transporte, bootstrap, relays, NAT extensions
- Overlay layer: rotas, DHT e persistência
- Service layer: mensagens, sessões e acesso a dados
- Application layer: PoC social e aplicações futuras

Camada superior consome abstrações; camada inferior fornece conectividade e transporte.

Mapa: superior consome; inferior fornece

Identidade: físico != virtual

A separação entre nó físico e identidade virtual é um dos diferenciais da arquitetura.

Physical Node

- processo real na rede
- endpoints e transporte
- conectividade física
- sessões físicas
- estado local e peers



Virtual Node

- identidade lógica
- perfil ou agente de aplicação
- chaves próprias
- rotas publicadas
- pode migrar sobre o físico

Distributed tables: coordenação entre camadas

As tabelas distribuídas separam descoberta física, rotas, dados, ponteiros e metadados.

DPNT

descoberta física e reachability

DRT

rotas para virtual nodes

DDT

holders de conteúdo

DPT

ponteiros mutáveis

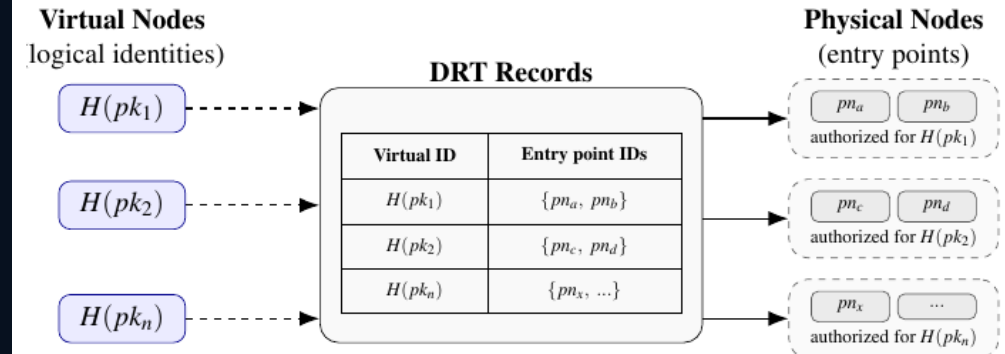
DTT

tags/metadados

Thus, the DRT does not store full hop sequences, nor does it prescribe the algorithm used to construct them. It only exposes the distributed information needed for a requester to start communication toward a reachable virtual identity [5], [8], [9].

Distributed Routing Table (DRT)

The DRT indirectly maps a virtual node identifier to physical entry points authorized to receive its traffic.



- The DRT does not store complete routes.
- It provides authorized physical entry points.
- Virtual-to-physical mapping remains indirect.

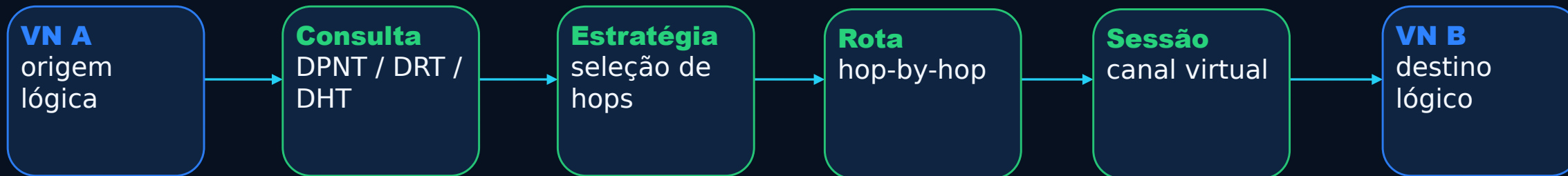
Exemplo: DRT publica reachability de virtual nodes

DHT-inspired coordination fabric
key = SHA512(namespace || logical_key)
responsabilidade por distância XOR; K = 3 no MVP

Tabelas: coordenação entre descoberta, rotas e dados

Da descoberta à comunicação virtual

Roteamento e sessão são tratados como fluxo arquitetural, não como função isolada.



Route protocol

define ciclo de vida: criação, publicação, validação e execução da rota.

Route Strategy

escolhe hops e critérios. No MVP, a estratégia principal é `random_walk_ttl`.

Fluxo: descobrir -> rotear -> sessão -> mensagem

Security model: autenticação, sigilo e crypto-agility

O MVP valida o pipeline de segurança, mas a arquitetura permite substituições criptográficas futuras.

Identificadores

SHA-512(public_key) para nós físicos e virtuais

Assinaturas

ML-DSA-65 quando disponível; autenticidade de registros

KEM

ML-KEM-768 para estabelecimento de segredo

AEAD

AES-256-GCM-SIV com AAD para payloads autenticados

Anti-abuso inicial

publication proof-of-work e validação de registros

Publication proof-of-work

Usado na escrita da DHT: caro de gerar, barato de validar. Nonce fica no payload; material canônico exclui pow_nonce.

Bibliotecas de crypto

OpenSSL CLI para ML-KEM/ML-DSA; cryptography para AES-256-GCM-SIV.

Segurança: identidade, assinatura, KEM e AEAD

MVP experimental: validação dos fluxos principais

O protótipo não é uma rede pública final; ele valida a arquitetura em ambiente controlado.

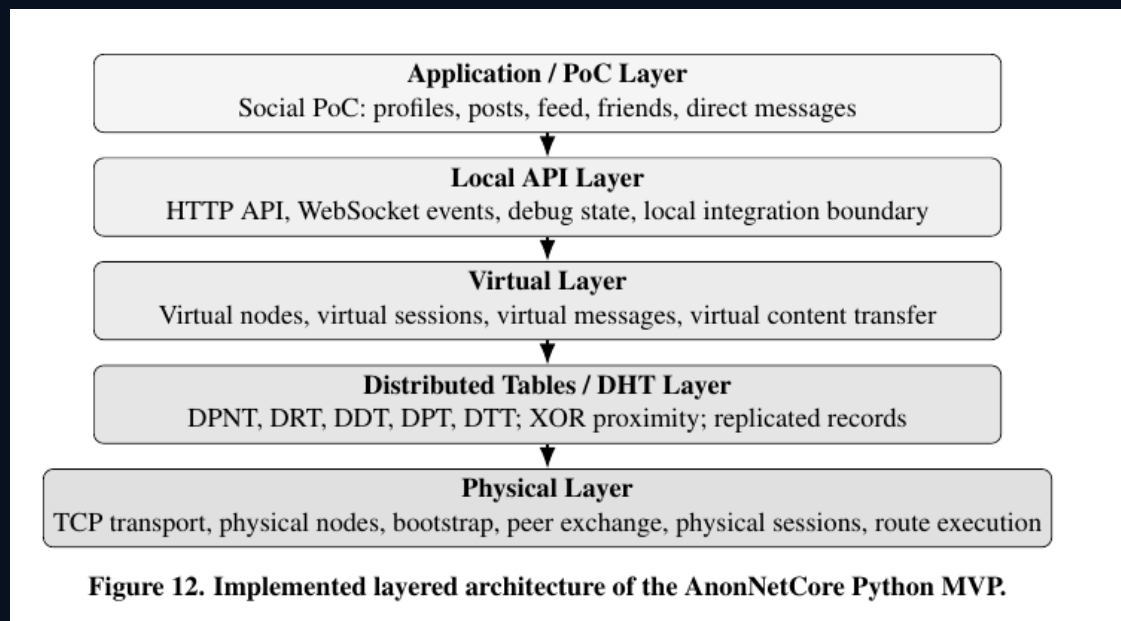


Figura 12: AnonNetCore Python MVP

Python MVP

TCP

UDP experimental

Relay-assisted forwarding

SQLite / SQLAlchemy

HTTP API / WebSocket

Social PoC

Docker/local clusters

Valida

bootstrap, descoberta, DHT, DRT, rotas, sessões virtuais, mensagens, conteúdo e API local.

Não promete

produção Internet-scale, NAT completo, auditoria criptográfica formal ou defesa anti-abuso madura.

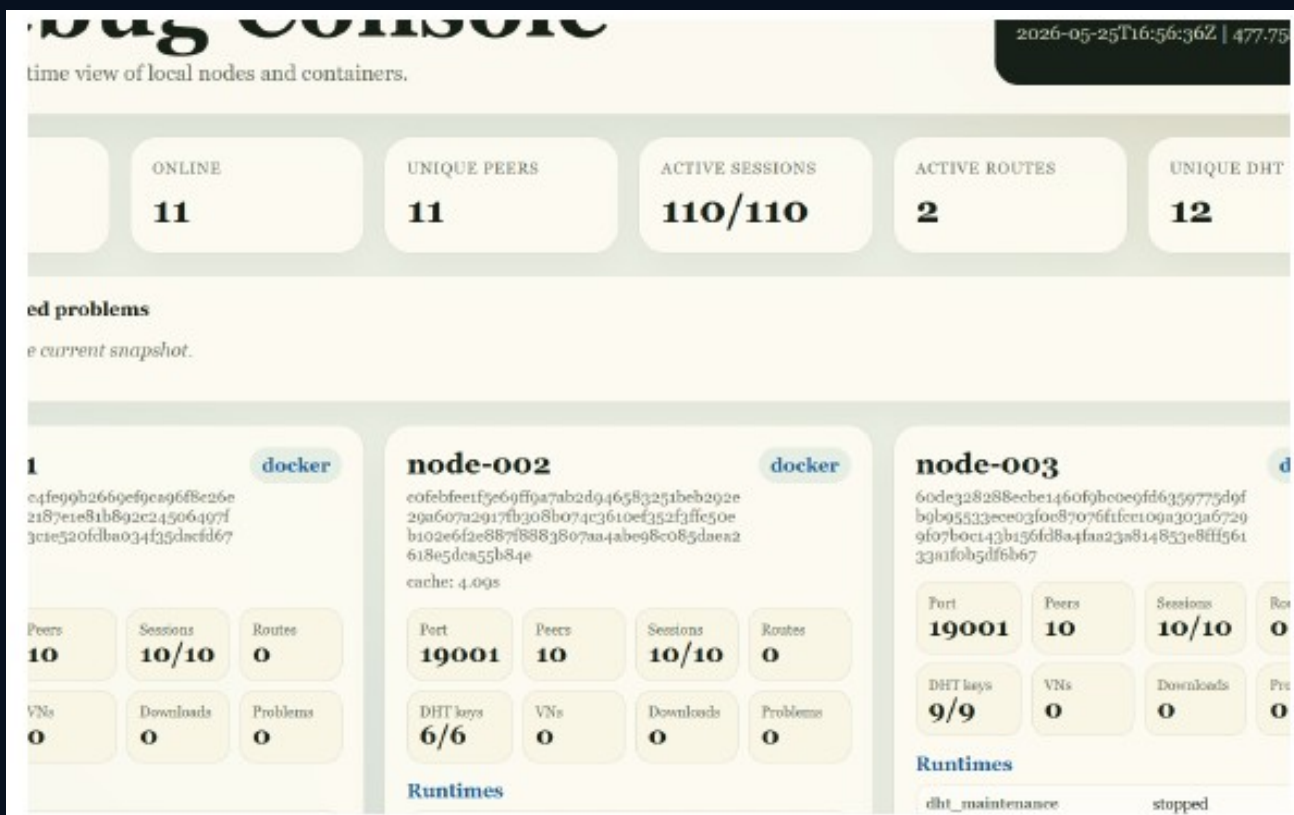
Stack implementado

Python 3.12 · asyncio · SQLAlchemy/SQLite · cryptography · websockets · OpenSSL CLI · Docker · HTML/CSS/JS

MVP: validar fluxos, não produto final

Fluxos implementados e observabilidade

O debug console e os endpoints locais ajudam a demonstrar estado real da execução.



Debug Console: observabilidade do experimento

Implemented flow

- Cluster local/Docker
- Bootstrap
- Peer validation
- DHT publish/query
- DRT resolution
- Virtual sessions
- Message/content transfer
- API + WebSocket + debug state

A validação exercita interações entre camadas implementadas, não apenas funções isoladas.

Prova: teste integra camadas reais

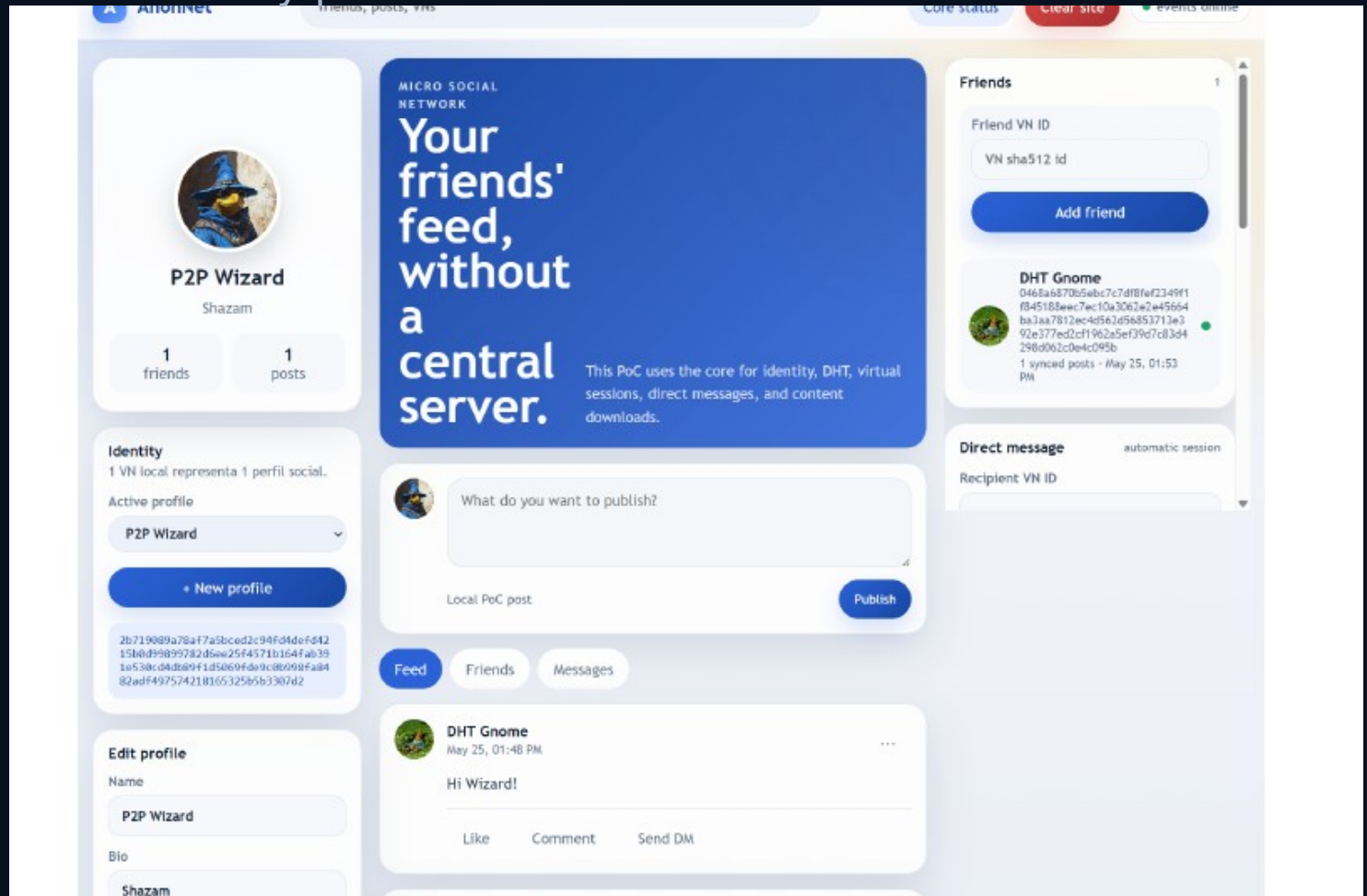
Social PoC: aplicação real sobre a camada

A aplicação demonstra consumo da camada overlay por meio da API local.

A PoC força a camada a resolver:

- criação de perfis
- publicação e resolução
- feed e sincronização
- mensagens diretas
- integração via API local

A rede social é o consumidor experimental da overlay, não o produto final.



Social PoC: perfis, feed e mensagens diretas

PoC: aplicação consumindo a overlay

Resultados observados

O MVP demonstrou o caminho funcional esperado para o protótipo.

14 · ATO 3 · RESULTADOS

**2/2
smoke tests
passed**

**20
nós em
cenário total**

**373.7s
execução
total**

Resultados funcionais

- Local physical cluster
- Virtual node creation
- DRT route publication
- Virtual session establishment
- Virtual message delivery
- Content advertisement and retrieval
- Social PoC integration

Leitura correta do resultado

Evidência funcional dos fluxos; não é medição de throughput, latência global ou rede pública.

```
(.venv) PS C:\Users\Anderson Simioni\Desktop\AnonNetCore_py> .\.venv\Scripts\python.exe ./scripts/run_smokes.py 10
Passed smokes:
OK core-full-flow          190.9s | core protocols, DHT/DRT, route, session, message, content
OK poc-full-flow          182.8s | social profiles, feed, DHT, DM, API flow

Logs: C:\Users\Anderson Simioni\Desktop\AnonNetCore_py\data\local\smoke-runs\20260608-143157

Network topology summary:
TOPOLOGY source=random draw per smoke cluster
TOPOLOGY total_cluster_instances=2 total_nodes=20
TOPOLOGY profiles={'bootstrap_public_tcp': 4, 'public_tcp_only': 9, 'private_relay_client': 5, 'public_tcp_u
dp': 2}
TOPOLOGY reachability={'public': 15, 'private': 5}
TOPOLOGY transports={'tcp_only': 13, 'tcp_udp': 7}
TOPOLOGY relay={'relay_capable': 15, 'not_relay_capable': 5}

Reported node warnings/errors:
REPORT total=2107 | file=reported-errors.jsonl
REPORT levels={'WARNING': 2107}
REPORT top_nodes={'node-001': 899, 'node-002': 405, 'asimioni': 213, 'node-003': 114, 'node-009': 97}
REPORT top_components={'physical_session_client': 783, 'engine': 598, 'physical_ping_client': 208, 'dht': 13
5, 'physical_dht_client': 106}
(.venv) PS C:\Users\Anderson Simioni\Desktop\AnonNetCore_py>
```

Evidência final dos smoke tests

Interpretação

O MVP demonstrou o caminho funcional completo esperado para o protótipo: da execução de nós físicos até a aplicação consumindo a camada.

Resultado: o caminho completo funcionou

Limitações assumidas do MVP

Os limites definem a fronteira entre protótipo experimental e rede pública robusta.

AnonNetCore é um MVP funcional: valida arquitetura e protocolos, mas não é produto de produção.

Security limits

sem defesa completa contra Sybil/flooding; sem auditoria formal; chaves locais e API local ainda não hardenizadas

Network limits

NAT traversal completo, STUN/TURN, QUIC e relay network madura ainda são futuros

DHT / route limits

sem Kademlia completa; conflitos e churn simples; random_walk_ttl é a estratégia principal; correlação global não resolvida

Reliability / PoC limits

confiabilidade leve; sem congestion control completo; feed, mídia, paginação, moderação e busca ainda simplificados

Limites adicionais importantes

key storage simples · sem local auth/hardening de API · sem Sybil hardening completo · sem congestion control completo · sem frame, usa JSON

A maturidade da pesquisa também está em declarar corretamente o que ela não resolve.

Roadmap técnico pós-MVP

16 · ATO 4 · FUTURO

O faults.md vira plano de evolução técnico, não apenas lista de problemas.

1

Conectividade

UDP hole punching, STUN/TURN, relay hardening e QUIC

2

DHT e churn

buckets, lookup iterativo, refresh, conflitos e sharding

3

Segurança pública

rate limits, scoring, Sybil hardening, local auth e auditoria

4

Escala e aplicação

testes longos, churn, perda, media objects, feeds paginados

Prioridade de evolução: primeiro conectividade e hardening; depois escala pública e recursos de aplicação.

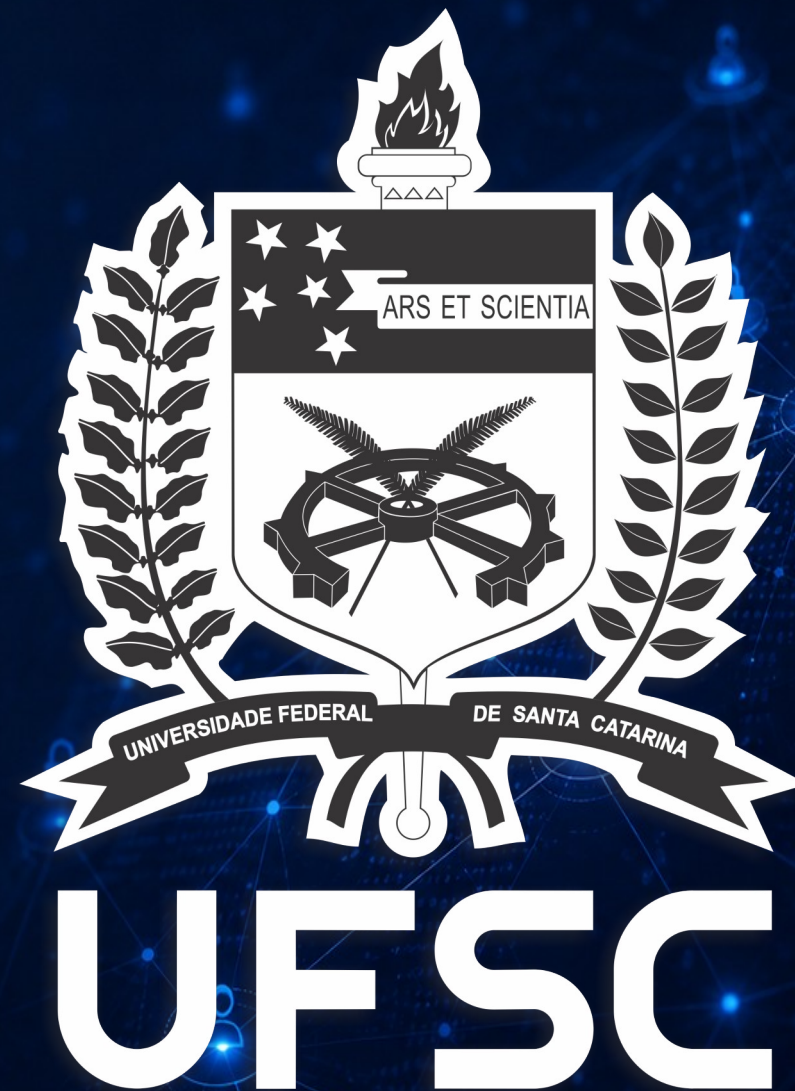
Futuro: conectar, escalar, endurecer, auditar

Conclusão

- 01 **Arquitetura modular para overlay P2P**
- 02 **Separação entre físico, virtual, rotas, sessões e dados**
- 03 **Integração de segurança, persistência e descoberta**
- 04 **Validação experimental com MVP e Social PoC**

A principal contribuição não é afirmar que a rede está pronta para produção, mas demonstrar uma base arquitetural consistente para sistemas descentralizados seguros, modulares e evolutivos.

arquitetura validada + limites honestos



Obrigado.

Perguntas

Pontos de defesa para a discussão com a banca.

18 · DISCUSSÃO

Anonimato

orientado à privacidade; sem prova formal contra adversário global

NAT traversal

previsto na arquitetura; MVP usa TCP, UDP experimental e relay

Kademlia

DHT inspirada em XOR/proximidade; ainda não é Kademlia completa

Escala

validação funcional e arquitetural; Internet-scale fica como futuro

Objetivo: validar a arquitetura e seus fluxos principais, não entregar uma rede pública madura.

O escopo e a arquitetura